🐣**Practical 1**
Title - Breadth First Search and Iterative Depth First Search

Aim - 1. To implement the Breadth First Search (BFS) algorithm to solve a given problem.

2. To implement the Iterative Depth First Search (DFS) algorithm to solve the same problem.

3. To compare the performance and efficiency of both algorithms.

Program 1: Breadth First Search (BFS)

```
graph = {
   '5': ['3', '7'],
   '3': ['2', '4'],
   '7': ['8'],
   '2': [],
   '4': ['8'],
   '8': []
}

visited = []
q = []

def bfs(graph, visited, node):
   visited.append(node)
   q.append(node)

   while q:
      m = q.pop(0)
      print(m)

      for n in graph[m]:
         if n not in visited:
            visited.append(n)
            q.append(n)

bfs(graph, visited, '5')
```

Program 2: Iterative Depth First Search (DFS)

```
graph = {
   '5': ['3', '7'],
```

```
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

visited = set()

def dfs(graph, start):
    stack = [start]

    while stack:
        node = stack.pop()

        if node not in visited:
            print(node)
            visited.add(node)

            for neighbor in reversed(graph[node]):
                if neighbor not in visited:
                    stack.append(neighbor)

dfs(graph, '5')
```

🐣Practical 2
Title - A* Search Algorithm

Aim - 1. To implement the A* search algorithm for solving a path-finding problem.
2. To compare the performance and effectiveness of the algorithm.

```
import heapq

# Romania map: cities and distances
romania_map = {
    'Arad': {'Zerind': 75, 'Timisoara': 118, 'Sibiu': 140},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99,
              'Rimnicu Vilcea': 80},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
```

```
    'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97,
                      'Craiova': 146},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146,
                'Pitesti': 138},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138,
                'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101,
                  'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98,
                 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}

# Heuristic values (straight-line distance to Bucharest)
heuristic = {
    'Arad': 366, 'Zerind': 374, 'Timisoara': 329,
    'Sibiu': 253, 'Oradea': 380, 'Lugoj': 244,
    'Mehadia': 241, 'Drobeta': 242, 'Craiova': 160,
    'Rimnicu Vilcea': 193, 'Fagaras': 176,
    'Pitesti': 100, 'Bucharest': 0, 'Giurgiu': 77,
    'Urziceni': 80, 'Hirsova': 151, 'Eforie': 161,
    'Vaslui': 199, 'Iasi': 226, 'Neamt': 234
}

def a_star_search(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic[start], 0, start, [start]))
    best_cost = {start: 0}

    while open_list:
        f, g, city, path = heapq.heappop(open_list)

        if g != best_cost.get(city):
            continue

        if city == goal:
            return path, g
```

```
        for neighbor, distance in graph[city].items():
            new_cost = g + distance

            if new_cost < best_cost.get(neighbor, float('inf')):
                best_cost[neighbor] = new_cost
                new_f = new_cost + heuristic[neighbor]

                heapq.heappush(
                    open_list,
                    (new_f, new_cost, neighbor, path + [neighbor])
                )

    return None, float('inf')


# Example usage
start_city = 'Arad'
goal_city = 'Bucharest'

path, cost = a_star_search(
    romania_map, heuristic, start_city, goal_city
)

if path:
    print("Path found:", " -> ".join(path))
    print("Total distance:", cost, "km")
else:
    print("No path found.")
```

🐣Practical 3
Title - Decision Tree Learning

Aim - 1.To implement the Decision Tree Learning algorithm for a given dataset.
2.To evaluate the accuracy and effectiveness of the decision tree on test data.
3.To visualize and interpret the generated decision tree.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris(as_frame=True)
data = iris.frame

# Separate features and target
X = data.drop("target", axis=1)
y = data["target"]

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Create and train the decision tree
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

# Predict the test data
y_pred = clf.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Visualize the decision tree
plt.figure(figsize=(12, 8))

plot_tree(
    clf,
    filled=True,
    feature_names=X.columns,
    class_names=iris.target_names,
    rounded=True
)

plt.title("Decision Tree Visualization")
plt.show()
```

🐣**Practical 4**
Title - Support Vector Machine

Aim -1.To implement the SVM algorithm for binary classification.
2.To train an SVM model using a given dataset.
3.To evaluate the model's performance on test data.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_breast_cancer

# Load the dataset
data = load_breast_cancer(as_frame=True).frame

# Separate features and target
X = data.drop("target", axis=1)
y = data["target"]

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create and train the SVM model
svm_classifier = SVC(kernel="linear")
svm_classifier.fit(X_train, y_train)

# Predict the test data
y_pred = svm_classifier.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Actual output:", list(y_test))
print("Predicted output:", list(y_pred))
print(f"Accuracy: {accuracy:.2f}")
```

🐣**Practical 5**
Title - Naive Bayes Classifier

Aim - 1.To implement the Naive Bayes algorithm for classification.
2.To train a Naive Bayes model using a dataset.
3.To evaluate the model using test data and calculate accuracy.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = load_iris()

# Separate features and target
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Create and train the Naive Bayes classifier
clf = GaussianNB()
clf.fit(X_train, y_train)

# Predict the test data
y_pred = clf.predict(X_test)

# Display actual and predicted outputs
print("Actual output:", y_test)
print("Predicted output:", y_pred)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

🐣**Practical 6**
Title - K-nearest Neighbour (KNN)

Aim - 1.To implement the KNN algorithm for classification.
2.To apply KNN to a dataset and predict the class of test data.
3.To evaluate the accuracy of the predictions.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

# Load the dataset
iris = load_iris()

# Create feature and target arrays
X = iris.data
y = iris.target

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Create the KNN classifier
knn = KNeighborsClassifier(n_neighbors=7)

# Train the model
knn.fit(X_train, y_train)

# Predict the test data
y_pred = knn.predict(X_test)

# Display actual and predicted outputs
print("Actual output:", y_test)
print("Prediction:", y_pred)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

🐣**Practical 7**
Title- Association Rule Mining

Aim - To identify frequent itemsets and generate meaningful association rules from a set of market-basket transactions using Python and the mlxtend library.

Terminal - pip install pandas mlxtend

```
from mlxtend.frequent_patterns import apriori, association_rules
import pandas as pd

# Sample transaction dataset
dataset = [
    ['milk', 'bread', 'nuts'],
    ['milk', 'bread'],
    ['milk', 'eggs', 'nuts'],
    ['milk', 'bread', 'eggs'],
    ['bread', 'nuts']
]

# Convert dataset to a pandas DataFrame
df = pd.DataFrame(dataset)

# Perform one-hot encoding
df_encoded = pd.get_dummies(
    df, prefix='', prefix_sep=''
)

# Find frequent itemsets using Apriori
frequent_itemsets = apriori(
    df_encoded,
    min_support=0.5,
    use_colnames=True
)

# Print frequent itemsets
print("Frequent Itemsets:")
print(frequent_itemsets)

# Generate association rules
rules = association_rules(
    frequent_itemsets,
```

```
    metric="lift",
    min_threshold=1.0
)

# Print association rules
print("\nAssociation Rules:")
print(rules[
    ['antecedents', 'consequents',
     'support', 'lift', 'leverage']
])
```

🐣Practical 8

Title - Adaboost Classifier

Aim - 1.To implement the AdaBoost classification algorithm using Python.
2.To train an AdaBoost classifier on the Iris dataset.
3.To predict the class labels for test data and evaluate the model's accuracy.

Terminal - pip install scikit-learn

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn import metrics

# Load Iris dataset
iris = datasets.load_iris()

# Separate features and target
X = iris.data
y = iris.target

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create AdaBoost classifier
abc = AdaBoostClassifier(
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
```

```
)

# Train the AdaBoost classifier
model = abc.fit(X_train, y_train)

# Print actual output
print("Actual output:", y_test)

# Predict the response for test dataset
y_pred = model.predict(X_test)
print("Prediction:", y_pred)

# Calculate and print accuracy
accuracy = metrics.accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```